TUANGOU

Vâ•- · Community Group-Buying Platform

Developer Onboarding Guide

For technical and non-technical team members

Next.js 15	Supabase	65 Tests	EN + ZH
Framework	Database + Auth	All Passing	Bilingual

v0.1.0 · June 2026

Topic	Details
What it is	Bilingual community group-buying e-commerce platform
Languages	English + Simplified Chinese (toggle in the UI)
Users	Shoppers, Guests, and Admins
Frontend	Next.js 15 · React 19 · TypeScript · Tailwind CSS
Backend	Next.js API Routes (/api/v1/*)
Database	Supabase (PostgreSQL)
Auth	Supabase Auth — email/password, Google, WeChat (coming soon)
Tests	65 unit tests — Vitest + React Testing Library
Run locally	npm install !' .env.local !' npm run seed !' npm run dev
Admin access	Manual SQL: UPDATE profiles SET role='admin' WHERE user_id='...'

1. What Is This Product?

In Plain English

Tuangou (Vâ•-, "group buying") is an online grocery store built for local communities. The core idea is simple: when neighbours shop together, everyone gets a better price.

Instead of each person placing a separate order and paying for home delivery, community members browse the same catalogue, add items to their carts, and collect their orders at a designated community pickup point — cutting costs for everyone. Think of it like a neighbourhood co-op, but online.

The platform is designed for multilingual communities in Canada (particularly BC), so every page, label, and error message is available in both English and Simplified Chinese. Shoppers can flip between the two with a single click.

Who Uses It?

Role	What They Do
Shoppers	Browse products, add items to cart, check out, and track orders
Guests	Browse freely without an account — must sign in to place an order
Admins	Manage products, review orders, see analytics, monitor activity

What Problem Does It Solve?

- Reduces per-order delivery costs through community pickup
- Unlocks bulk-buy discounts when enough neighbours buy together
- Serves communities whose first language is Chinese (full bilingual support)
- Provides fresh, locally-sourced goods with no artificial preservatives

> *Think of the tech stack as the collection of tools and building materials the team chose. Just as a builder selects timber, bricks, and power tools, developers choose frameworks, databases, and libraries.*

Layer	Technology	What It Does
UI Framework	Next.js 15 (App Router)	Powers every page — storefront and admin dashboard
UI Language	React 19 + TypeScript	Component system; TypeScript prevents common bugs
Styling	Tailwind CSS 3.4	Utility-first CSS for responsive, consistent UI
UI Primitives	Radix UI	Accessible building blocks (dialogs, dropdowns, sliders)
Icons	Lucide React	Clean icon set used throughout the interface
Charts	Recharts	Admin dashboard revenue and order trend charts
Forms	React Hook Form + Zod	Form state management and validation rules
Database	Supabase (PostgreSQL)	Products, orders, users, carts, activity logs
Authentication	Supabase Auth	Manages sign-up, sign-in, and sessions
API	Next.js API Routes	Server endpoints the browser calls to read/write data
Testing	Vitest + RTL	Automated tests that catch regressions
Test Mocking	MSW	Intercepts HTTP in tests — no real network needed

Key Configuration Files

File	Purpose
.env.local	Secret keys and feature flags — NEVER commit this file
next.config.ts	Next.js settings (allowed image domains)
tailwind.config.ts	Custom colour palette and animations
tsconfig.json	TypeScript compiler settings (strict mode, path aliases)
vitest.config.mts	Test runner configuration
package.json	All dependencies and runnable scripts

3. Project Structure

Everything lives in the src/ directory. Here's the map:

```
src/
% % %  app/                      !• Next.js pages and API routes
%    % % %  (storefront)/        !• Public-facing pages (home, shop, cart, checkout…)
%    % % %  admin/               !• Admin dashboard (restricted to admin role)
%    % % %  api/v1/              !• REST API endpoints the browser calls
%    % % %  auth/callback/       !• OAuth redirect handler
%
% % %  components/
%    % % %  ui/                  !• Low-level primitives (Button, Card, Dialog…)
%    % % %  common/              !• Shared layout (Navbar, Footer, LanguageToggle)
%    % % %  storefront/          !• Product components (ProductCard, Hero, Filter…)
%    % % %  admin/               !• Dashboard components (AdminGate, ProductForm…)
%
% % %  providers/               !• React Contexts: Auth, Cart, Language
% % %  lib/
%    % % %  repository.ts        !• Data layer — all API calls go through here
%    % % %  api/client.ts        !• HTTP wrapper (adds auth header, timeouts)
%    % % %  storage.ts           !• localStorage read/write helpers
%    % % %  utils.ts             !• Shared helpers (formatPrice, formatDate…)
%
% % %  i18n/                    !• All bilingual text strings + LanguageProvider
% % %  data/                    !• Static seed data (36 products, 8 categories)
% % %  types.ts                 !• Shared TypeScript types (Product, Order…)
% % %  test/                    !• Test setup, MSW server, Supabase mock
```

How the Three Main Sections Relate

```
Browser visits /shop
      !“
(storefront)/shop/page.tsx         !• React page component
      !“ calls
lib/repository.ts !’ listProducts() !• Data layer
      !“ calls
/api/v1/products                   !• Next.js API route
      !“ queries
Supabase (PostgreSQL)              !• Database
      !‘ returns rows !’ mapped to Product[] !’ page renders
```

4. How Authentication Works

> *Plain-English summary: Tuangou uses Supabase Auth to handle identity. Sign-in is supported via email/password and Google. A WeChat option exists in the code but is currently disabled. Once signed in, the app remembers the user via a JWT token (a secure string stored in a cookie) that is automatically attached to every API request.*

The Complete Sign-In Flow

```
1. User visits /account/login
   % %  Options: Email/password  |  Google  |  WeChat (disabled)

2. User submits email + password
   % %  AuthProvider.signInWithPassword()
   % %  !' supabase.auth.signInWithPassword()
   % %  Supabase validates !' returns a JWT session

3. Session stored in a cookie
   % %  middleware.ts refreshes it on every page visit

4. AuthProvider fetches GET /api/v1/me
   % %  Returns: { role, name, email, phone, wechat_id, avatar_url }
   % %  Sets user state across the whole app

5. User is logged in — cart merges, orders become accessible
```

Roles & Permissions

Role	Who Has It	What They Can Access
user	Everyone who signs up (default)	Browse, buy, manage own orders and profile
admin	Manually promoted via SQL	Everything above + full admin dashboard

> *Promoting someone to admin requires one SQL query in Supabase. There is no UI for this yet. See §8 for step-by-step instructions.*

API Security

Every /api/v1/* endpoint checks the Authorization: Bearer <token> header. Three helper functions handle this:

Helper	What It Does
getAuthUser(req)	Reads the token, validates it with Supabase, returns { userId, role }
requireAuth(req)	Same — returns HTTP 401 if no valid token is present
requireAdmin(req)	Same — returns HTTP 403 if the user's role isn't 'admin'

Cart & Session Isolation

When a user signs out, their cart is immediately cleared — both from memory and from localStorage — so the next person using the same device does not see someone else's items. When a user signs in, any guest cart items are automatically merged into their server-side cart.

The Architecture in One Diagram

```
BROWSER                         NEXT.JS SERVER              SUPABASE
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %   % % % % % % % % % % % % % % % % % % % % % %
React Component
   !“ calls
lib/repository.ts
   !“ fetch() + JWT token
                                /api/v1/* route handler
                                  !“ requireAuth() checks token
                                  !“ supabase admin client
                                                              DB query
                                                              !‘ rows
                                  !‘ maps rows !’ DTOs
                                  !‘ JSON response
   !‘ typed result (Product, Order…)
Component re-renders
```

Example: Placing an Order

```
1. CHECKOUT FORM (browser)
   User fills in: name, phone, address, community pickup
   Zod validates all required fields

2. SUBMIT  !’  POST /api/v1/orders
   Body: { items: [{productId, quantity}], name, phone… }
   Header: Authorization: Bearer <jwt>

3. SERVER VALIDATES
   '  requireAuth() — confirms a valid logged-in user
   '  Fetches ALL product prices from the DATABASE
     (client-supplied prices are ignored — prevents tampering)
   '  Checks every productId exists — 409 if any are missing
   '  Calculates totals server-side
   '  Generates a unique order ID (idempotency-safe)

4. DATABASE WRITES
   INSERT orders + order_items
   DELETE cart_items (cart is cleared)
   INSERT activities (type='PLACE_ORDER')

5. RESPONSE: 201 Created + full Order object

6. BROWSER: clear cart !’ redirect to /order/confirmed/[id]
```

> *Security: Prices are always read from the database at checkout — the browser never sets the price. A malicious user cannot modify the request body to pay less.*

Storefront (Public — No Account Required)

Page	What It Does
Home (/)	Hero banner, popular products carousel, new arrivals, category tiles, trust bar
Shop (/shop)	Full catalogue with price/dietary filters and sort options
Category (/category/[slug])	Same as Shop, pre-filtered to one category
Product Detail (/product/[slug])	Images, description, dietary tags, stock status, Add to Cart, info tabs
Cart (/cart)	Line items with quantity controls, subtotal, delivery fee, total
Checkout (/checkout)	Contact info, community pickup, address, payment section
Order Confirmed (/order/confirmed/[id])	Confirmation page with order ID and summary

Account (Sign-In Required)

Page	What It Does
Login (/account/login)	Email/password form, Google OAuth button, WeChat button (disabled)
Account Home (/account)	Welcome screen, links to orders, saved items, sign out
Order History (/account/orders)	All past orders, expandable to see line items and status

Admin Dashboard (Requires admin Role)

Page	What It Does
Dashboard (/admin)	Stats cards, 7-day orders trend chart, revenue by category chart
Orders (/admin/orders)	All orders with inline status editing (Confirmed !' Processing !' Completed)
Products (/admin/products)	Full catalogue with create/edit dialog and delete confirmation
Users (/admin/users)	All users with email, phone, order count, total spent
Activity (/admin/activity)	Chronological log of all actions: sign-ins, cart adds, orders, edits

> *Plain-English summary: Think of each database table as a spreadsheet. Tables are linked together through shared IDs — just like a pivot table in Excel.*

Tables at a Glance

Table	What It Stores
profiles	Extra info about each user (name, phone, WeChat ID, role)
categories	The 8 product categories (Fresh Meat, Eggs & Dairy, Snacks…)
products	The full product catalogue (name EN+ZH, price, stock, dietary tags)
carts	One cart row per signed-in user
cart_items	The individual products in each cart (product ID + quantity)
orders	Every placed order (subtotal, delivery fee, status, address)
order_items	Line items within each order (price at time of purchase + qty)
activities	Immutable audit log of every user action

Relationships

```
auth.users %  1:1 % % %¶ profiles
auth.users %  1:1 % % %¶ carts %  1:many % % %¶ cart_items % % %¶ products
auth.users %  1:many % % %¶ orders %  1:many % % %¶ order_items % % %¶ products
products % % %¶ categories
auth.users %  1:many % % %¶ activities
```

Important Details

- Prices are stored as integer cents — 1999 means $19.99 (avoids floating-point rounding errors)
- Order items snapshot the price at purchase time — if a price changes later, old orders are unaffected
- Dietary tags are stored as comma-delimited text — e.g., 'VEGAN,ORGANIC'
- cart_items has a unique constraint on (cart_id, product_id) — quantities are merged, not duplicated
- Run npm run seed once per fresh Supabase project to load categories and products

Prerequisites

- Node.js 18 or newer — check with: node --version
- A Supabase project (free tier works) — app.supabase.com
- The repository cloned to your machine

Step-by-Step

Step 1 — Install Dependencies

```
npm install
```

Step 2 — Create Your Environment File

Create a file called .env.local in the project root:

```
NEXT_PUBLIC_WECHAT_ENABLED=false

# From your Supabase project !’ Settings !’ API
NEXT_PUBLIC_SUPABASE_URL=https://YOUR_PROJECT_ID.supabase.co/
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_...
SUPABASE_SERVICE_ROLE_KEY=sb_secret_...
```

> *Finding these values: In the Supabase dashboard !’ Settings !’ API. 'Project URL' is SUPABASE_URL, the 'anon public' key is PUBLISHABLE_KEY, and 'service_role' key is SERVICE_ROLE_KEY.*

Step 3 — Seed the Database

Loads 8 categories and 36 products into your Supabase project. Only needs to run once.

```
npm run seed
```

Step 4 — Start the Development Server

```
npm run dev
# Visit http://localhost:3000
```

Step 5 — Verify Everything Is Working

```
curl http://localhost:3000/api/v1/health
# Expected: {"status":"ok","supabase":{"reachable":true,...}}
```

> *If you see a 500 error mentioning SUPABASE_SERVICE_ROLE_KEY, double-check your .env.local file.*

Step 6 — Promoting Yourself to Admin (Optional)

Open the Supabase dashboard !’ SQL Editor and run:

```
-- Find your user ID
SELECT id, email FROM auth.users;

-- Promote yourself
UPDATE profiles SET role = 'admin'
WHERE user_id = 'paste-your-uuid-here';
```

9. Testing

Then sign in and visit http://localhost:3000/admin

```

```

Running Tests

```
npm test              # Run all 65 tests once
npm run test:watch    # Re-runs on every file save
```

What's Tested

Category	Files	Approx. Count
API route handlers	api/__tests__/*.test.ts	~40 tests
React components	providers/__tests__/*.tsx, admin/ tests /*.tsx	~15 tests
Repository / data layer	lib/__tests__/repository.test.ts	~10 tests

How the Test Setup Works

Tests run in a simulated browser environment (jsdom) — no real browser needed. All HTTP calls are intercepted by MSW (Mock Service Worker), which returns fixture data instead of hitting a real server. Supabase is fully mocked so tests don't need a database connection.

```
Test runs
  !“ renders React component
  !“ component calls fetch() or supabase.*
  !“ MSW intercepts !’ returns fixture JSON
  !“ assertion checks the rendered output
```

10. Internationalisation (EN/ZH)

How it works

All visible text is stored in a flat dictionary at src/i18n/strings.ts. Every key has both an English and a Chinese value:

```
"product.addToCart": { en: "Add to Cart", zh: "R Qe•-ri•f" },
"nav.shop":          { en: "Shop",         zh: "UF^—" },
```

Components access strings via the useLanguage() hook:

```
const { t, locale } = useLanguage();
<button>{t("product.addToCart")}</button>
// Renders "Add to Cart" (EN) or "R Qe•-ri•f" (ZH) depending on locale
```

The toggle pill in the navbar switches locales instantly. The preference is saved in localStorage so it persists between visits.

Adding New Text

- Add a new key to src/i18n/strings.ts with both en and zh values
- Use t("your.new.key") in your component
- If the key is missing, a warning is logged in development mode
- Price formatting adapts to locale: $19.99 (EN) vs ¥19.99 (ZH)

The project is an MVP — it works end-to-end, but some features are intentionally incomplete:

Feature	Status	Notes
Search	Not implemented	Search bar is visible but does nothing
Saved Items (Wishlist)	Not implemented	Link exists in the account menu
WeChat OAuth	Disabled	Code exists; set NEXT_PUBLIC_WECHAT_ENABLED=true when ready
Product image upload	URL only	Admin form accepts a URL; no file picker
Email notifications	Not implemented	No order confirmation email sent
Admin role promotion UI	Not implemented	Requires manual SQL (see §8)
Payment processing	Visual only	Checkout form does not charge anyone
Pagination	Not implemented	All data loads at once (fine for MVP scale)

Seed Data Note

> *The static data file (src/data/products.ts) has 74 products, but only 36 are seeded to the database. Products not in the database will return a 409 error at checkout. Always run npm run seed after setting up a new Supabase project.*

Appendix — Useful Commands & Environment Variables

All Available Commands

Command	What It Does
npm run dev	Start local development server at http://localhost:3000
npm test	Run all 65 tests once
npm run test:watch	Run tests in watch mode (re-runs on save)
npm run lint	Run ESLint across the codebase
npx tsc --noEmit	TypeScript type-check (no output files generated)
npm run seed	Seed categories and products into Supabase
npm run build	Build for production
npm start	Start the production server

Environment Variables Reference

Variable	Required	Where to Find It
NEXT_PUBLIC_SUPABASE_URL	Yes	Supabase !’ Settings !’ API !’ Project URL
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	Yes	Supabase !’ Settings !’ API !’ anon public kev
SUPABASE_SERVICE_ROLE_KEY	Yes (server only)	Supabase !’ Settings !’ API !’ service_role kev
NEXT_PUBLIC_WECHAT_ENABLED	No	Set to true only when WeChat OAuth is fully configured

> *SECURITY: SUPABASE_SERVICE_ROLE_KEY bypasses Row-Level Security — it is server-side only and must NEVER be exposed to the browser. Variables prefixed with NEXT_PUBLIC_ are visible in the browser bundle — never put secrets there.*